

Academic Knowledge-Sharing Platform

Final Optimized Architecture - NestJS + FastAPI

Version: 3.0 (NestJS + FastAPI Stack)
Last Updated: January 2026
Status: Production Ready

📄 Table of Contents

- [1. Executive Summary](#)
- [2. Technology Stack \(NestJS + FastAPI\)](#)
- [3. System Architecture](#)
- [4. Database Design](#)
- [5. API Architecture \(REST + GraphQL\)](#)
- [6. Developer Workload Distribution \(50/50\)](#)
- [7. Development Timeline \(Rebalanced\)](#)
- [8. Security Architecture](#)
- [9. NestJS Implementation Details](#)
- [10. FastAPI Implementation Details](#)
- [11. Deployment & Scaling](#)
- [12. Phase 1 MVP Launch Strategy](#)

Executive Summary

What We're Building

An **academic knowledge-sharing platform** where students can:

- Upload and discover study notes
- Ask and answer questions
- Earn reputation through contributions
- Engage in real-time discussions
- Build a collaborative learning community

Technology Stack Decision: NestJS + FastAPI

Why NestJS (replacing Node.js/Express)?

- ✔ **Enterprise-grade** - Built for large-scale applications
- ✔ **TypeScript-first** - Full type safety across backend
- ✔ **Dependency Injection** - Clean, testable, maintainable code
- ✔ **Modular Architecture** - Clear separation of concerns
- ✔ **Built-in GraphQL** - First-class GraphQL support with decorators
- ✔ **WebSocket Support** - Native real-time capabilities
- ✔ **Microservices Ready** - Easy to split if needed
- ✔ **Excellent Documentation** - Best-in-class docs and community
- ✔ **Testing Built-in** - Jest integration out of the box

Why FastAPI (replacing Flask)?

- ✔ **Modern Python** - Python 3.10+ with type hints
- ✔ **Async by Default** - Better performance than Flask
- ✔ **Auto API Docs** - Swagger/OpenAPI auto-generated
- ✔ **Type Safety** - Pydantic models for validation
- ✔ **Fast Performance** - One of the fastest Python frameworks
- ✔ **Easy to Learn** - Simple, intuitive syntax
- ✔ **GraphQL Support** - Strawberry GraphQL integration
- ✔ **Dependency Injection** - Clean architecture patterns

✔ **WebSocket Support** - Built-in async WebSocket

Architecture Highlights

- ✔ **50/50 Workload** - Equal distribution between FastAPI and NestJS devs
- ✔ **Hybrid API** - REST for CRUD, GraphQL for complex queries & real-time
- ✔ **Type-Safe Stack** - TypeScript (NestJS) + Python Type Hints (FastAPI)
- ✔ **Phase 1 MVP** - Fully launchable in 6 weeks
- ✔ **Enterprise Patterns** - DI, modules, decorators, clean architecture
- ✔ **Production Ready** - Monitoring, logging, testing, CI/CD

Technology Stack

Complete Stack Comparison

Component	Old Stack	New Stack	Why Change?
Core API (Python)	Flask	FastAPI	Async, faster, auto-docs, type safety
Real-time API (Node)	Express + Apollo	NestJS	Enterprise patterns, DI, modular, TypeScript
Frontend	Next.js 14	Next.js 14	✔ No change - already perfect
Database	PostgreSQL 15	PostgreSQL 15	✔ No change - industry standard
Cache/Queue	Redis 7	Redis 7	✔ No change - best for caching
ORM (Python)	SQLAlchemy	SQLAlchemy 2.0	✔ Better async support
ORM (Node)	None	Prisma	Type-safe ORM, migrations, studio
GraphQL (Python)	Ariadne	Strawberry	Better FastAPI integration
GraphQL (Node)	Apollo Server	NestJS GraphQL	Built-in, decorator-based
Validation (Python)	Manual	Pydantic	Auto-validation, type safety
Validation (Node)	Zod	class-validator	NestJS native, decorator-based

Full Technology Stack

Backend - FastAPI (Python)

Framework:	FastAPI 0.104+
Python:	3.11+
ORM:	SQLAlchemy 2.0 (async)
Validation:	Pydantic v2
GraphQL:	Strawberry GraphQL
Auth:	python-jose (JWT)
Password:	passlib (Argon2)
Testing:	pytest + pytest-asyncio
API Docs:	Built-in (Swagger/OpenAPI)
CORS:	fastapi-cors

Backend - NestJS (TypeScript)

Framework:	NestJS 10+
Runtime:	Node.js 18 LTS
Language:	TypeScript 5+
ORM:	Prisma 5+
GraphQL:	@nestjs/graphql + Apollo
Validation:	class-validator + class-transformer
Auth:	@nestjs/jwt + passport
WebSocket:	@nestjs/websockets + socket.io
Queue:	@nestjs/bull + Bull
Testing:	Jest (built-in)
Cache:	@nestjs/cache-manager

Frontend - Next.js

Framework:	Next.js 14+ (App Router)
Language:	TypeScript 5+
Styling:	TailwindCSS 3+
UI:	shadcn/ui
State:	Zustand (global) + Apollo Client (GraphQL)
Forms:	React Hook Form + Zod
Real-time:	Apollo Subscriptions
File Upload:	Uppy
Rich Text:	Tiptap
PDF Viewer:	React-PDF

Infrastructure

Database:	PostgreSQL 15+
Cache/Queue:	Redis 7+
Storage:	AWS S3 + CloudFront
Search:	PostgreSQL Full-Text (MVP) → Elasticsearch (later)
Email:	Nodemailer (NestJS)
Virus Scan:	ClamAV
Monitoring:	Prometheus + Grafana
Logging:	Winston (NestJS) + Python logging
Errors:	Sentry
Container:	Docker + Docker Compose
CI/CD:	GitHub Actions
Cloud:	AWS / DigitalOcean

System Architecture

High-Level Architecture



Why This Architecture?

- 1. Type Safety Everywhere** - FastAPI: Python type hints + Pydantic validation - NestJS: TypeScript + decorators + DTOs - Frontend: TypeScript + Zod schemas - Database: Prisma (NestJS) + SQLAlchemy with types
- 2. Enterprise Patterns** - Dependency Injection (both FastAPI and NestJS) - Modular architecture (NestJS modules) - Clean separation of concerns - Easy to test and maintain
- 3. Auto-Generated Documentation** - FastAPI: Swagger UI at `/docs` , ReDoc at `/redoc` - NestJS: GraphQL Playground at `/graphql`

- 4. Performance** - FastAPI: Async by default, very fast - NestJS: Non-blocking I/O, optimized - PostgreSQL: Proper indexing - Redis: In-memory caching
- 5. Developer Experience** - Auto-completion everywhere (TypeScript + type hints) - Clear error messages - Hot reload in development - Integrated testing frameworks
-

Database Design

Database Schema (No Changes)

The database schema remains the same as before. Using PostgreSQL 15+ with the following core tables:

- **users** - Student accounts, authentication, reputation
- **courses** - Academic courses
- **notes** - Study materials (PDFs, images)
- **questions** - Student questions
- **answers** - Responses to questions
- **votes** - Upvote/downvote tracking (polymorphic)
- **tags** - Content categorization
- **comments** - Threaded discussions (polymorphic)
- **bookmarks** - Saved content
- **notifications** - In-app alerts
- **reports** - Content moderation

Full schema available in previous document (unchanged).

ORM Differences

FastAPI (SQLAlchemy 2.0)

```
# models/user.py
from sqlalchemy import Column, String, Integer, Boolean, DateTime
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import relationship
from database import Base
import uuid
from datetime import datetime

class User(Base):
    __tablename__ = "users"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    email = Column(String(255), unique=True, nullable=False, index=True)
    username = Column(String(50), unique=True, nullable=False, index=True)
    password_hash = Column(String(255), nullable=False)
    full_name = Column(String(100))
    reputation_score = Column(Integer, default=0, index=True)
    role = Column(String(20), default='user')
    is_verified = Column(Boolean, default=False)
    is_banned = Column(Boolean, default=False)
    created_at = Column(DateTime, default=datetime.utcnow)

    # Relationships
    notes = relationship("Note", back_populates="uploader")
    questions = relationship("Question", back_populates="author")
```

NestJS (Prisma)

```
// schema.prisma
model User {
  id          String    @id @default(uuid()) @db.Uuid
  email       String    @unique @db.VarChar(255)
  username    String    @unique @db.VarChar(50)
  passwordHash String    @map("password_hash") @db.VarChar(255)
  fullName    String?   @map("full_name") @db.VarChar(100)
  reputationScore Int    @default(0) @map("reputation_score")
  role        String    @default("user") @db.VarChar(20)
  isVerified  Boolean   @default(false) @map("is_verified")
  isBanned    Boolean   @default(false) @map("is_banned")
  createdAt   DateTime  @default(now()) @map("created_at")

  // Relations
  comments      Comment[]
  notifications  Notification[]

  @@index([email])
  @@index([username])
  @@index([reputationScore])
  @@map("users")
}
```

Key Differences: - Both ORMs connect to the same PostgreSQL database - FastAPI uses SQLAlchemy (Python standard, powerful) - NestJS uses Prisma (TypeScript-native, great DX) - Both support async operations - Both generate migrations

API Architecture

API Decision Matrix

Use Case	API Type	Framework	Why?
Authentication	REST	FastAPI	Standard OAuth2, simple flow
File Upload	REST	FastAPI	Multipart, presigned URLs
Notes CRUD	REST	FastAPI	Standard patterns, caching
Questions CRUD	REST	FastAPI	Standard patterns
Voting	REST	FastAPI	Simple operations
Comments	GraphQL	NestJS	Nested data, real-time
Notifications	GraphQL	NestJS	Real-time updates
Leaderboards	GraphQL	NestJS	Aggregated data
Real-time Updates	GraphQL Subscriptions	NestJS	WebSocket native

FastAPI Endpoints (REST)

Base URL: `http://localhost:8000/api/v1`

Authentication

```
from fastapi import APIRouter, Depends, HTTPException, status
```

```

from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from pydantic import BaseModel, EmailStr
from typing import Optional

router = APIRouter(prefix="/auth", tags=["Authentication"])

class UserRegister(BaseModel):
    email: EmailStr
    username: str
    password: str
    full_name: Optional[str] = None

class TokenResponse(BaseModel):
    access_token: str
    refresh_token: str
    token_type: str = "bearer"
    expires_in: int = 900 # 15 minutes

@router.post("/register", response_model=UserResponse)
async def register(data: UserRegister, db: AsyncSession = Depends(get_db)):
    """Register new user account"""
    # Check if email exists
    existing = await db.execute(
        select(User).where(User.email == data.email)
    )
    if existing.scalar_one_or_none():
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Email already registered"
        )

    # Hash password
    hashed = hash_password(data.password)

    # Create user
    user = User(
        email=data.email,
        username=data.username,
        password_hash=hashed,
        full_name=data.full_name
    )
    db.add(user)
    await db.commit()
    await db.refresh(user)

    # Send verification email (async task)
    background_tasks.add_task(send_verification_email, user.email)

    return user

@router.post("/login", response_model=TokenResponse)
async def login(
    form_data: OAuth2PasswordRequestForm = Depends(),
    db: AsyncSession = Depends(get_db)
):
    """Login and get JWT tokens"""
    # Find user
    result = await db.execute(
        select(User).where(User.email == form_data.username)
    )

```

```

)

user = result.scalar_one_or_none()

if not user or not verify_password(form_data.password, user.password_hash):
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Incorrect email or password"
    )

if user.is_banned:
    raise HTTPException(
        status_code=status.HTTP_403_FORBIDDEN,
        detail="Account is banned"
    )

# Generate tokens
access_token = create_access_token(user.id)
refresh_token = create_refresh_token(user.id)

return TokenResponse(
    access_token=access_token,
    refresh_token=refresh_token,
    expires_in=900
)

```

Notes Endpoints

```

from fastapi import APIRouter, UploadFile, File, Form
from typing import List, Optional

router = APIRouter(prefix="/notes", tags=["Notes"])

class NoteCreate(BaseModel):
    title: str
    description: Optional[str] = None
    course_id: UUID
    tags: List[str] = []
    is_anonymous: bool = False

class NoteResponse(BaseModel):
    id: UUID
    title: str
    description: Optional[str]
    file_url: str
    file_type: str
    upvotes: int
    view_count: int
    uploader: Optional[UserResponse]
    course: CourseResponse
    tags: List[TagResponse]
    created_at: datetime

class Config:
    from_attributes = True

@router.get("/", response_model=List[NoteResponse])
async def list_notes(
    course_id: Optional[UUID] = None,

```



```

tag: Optional[str] = None,
search: Optional[str] = None,
sort_by: str = "created_at",
page: int = 1,
limit: int = 20,
db: AsyncSession = Depends(get_db),
current_user: Optional[User] = Depends(get_current_user_optional)
):
    """List notes with filters and pagination"""
    query = select(Note)

    # Filters
    if course_id:
        query = query.where(Note.course_id == course_id)
    if tag:
        query = query.join(Note.tags).where(Tag.slug == tag)
    if search:
        query = query.where(
            Note.search_vector.match(search)
        )

    # Sorting
    if sort_by == "popular":
        query = query.order_by(Note.upvotes.desc())
    elif sort_by == "recent":
        query = query.order_by(Note.created_at.desc())

    # Pagination
    offset = (page - 1) * limit
    query = query.offset(offset).limit(limit)

    result = await db.execute(query)
    notes = result.scalars().all()

    return notes

@router.post("/", response_model=NoteResponse, status_code=status.HTTP_201_CREATED)
async def create_note(
    data: NoteCreate,
    file_key: str,
    db: AsyncSession = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    """Create a new note"""
    # Verify file exists in S3
    if not await verify_s3_file(file_key):
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="File not found"
        )

    # Calculate content hash
    content_hash = await get_file_hash(file_key)

    # Check for duplicates
    existing = await db.execute(
        select(Note).where(
            Note.course_id == data.course_id,
            Note.content_hash == content_hash

```

```

    )
)
if existing.scalar_one_or_none():
    raise HTTPException(
        status_code=status.HTTP_409_CONFLICT,
        detail="This file has already been uploaded to this course"
    )

# Create note
note = Note(
    title=data.title,
    description=data.description,
    course_id=data.course_id,
    uploader_id=current_user.id,
    file_url=get_cdn_url(file_key),
    file_type=get_file_type(file_key),
    file_size=await get_file_size(file_key),
    content_hash=content_hash,
    is_anonymous=data.is_anonymous
)
db.add(note)
await db.commit()
await db.refresh(note)

# Add tags
for tag_name in data.tags:
    tag = await get_or_create_tag(db, tag_name)
    note.tags.append(tag)
await db.commit()

# Trigger background jobs
await redis.publish('note_created', json.dumps({
    'note_id': str(note.id),
    'uploader_id': str(current_user.id)
}))

return note

@router.post("/{note_id}/vote")
async def vote_on_note(
    note_id: UUID,
    value: int, # 1 or -1
    db: AsyncSession = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    """Upvote or downvote a note"""
    if value not in [-1, 1]:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Vote value must be 1 or -1"
        )

    # Check if note exists
    note = await db.get(Note, note_id)
    if not note:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Note not found"
        )

```

```

# Check existing vote
result = await db.execute(
    select(Vote).where(
        Vote.user_id == current_user.id,
        Vote.target_id == note_id,
        Vote.target_type == "note"
    )
)

existing_vote = result.scalar_one_or_none()

if existing_vote:
    if existing_vote.value == value:
        # Remove vote
        await db.delete(existing_vote)
        note.upvotes -= value
    else:
        # Change vote
        existing_vote.value = value
        note.upvotes += (2 * value) # +2 or -2
else:
    # New vote
    vote = Vote(
        user_id=current_user.id,
        target_id=note_id,
        target_type="note",
        value=value
    )
    db.add(vote)
    note.upvotes += value

await db.commit()

# Publish event for real-time updates
await redis.publish('vote_updated', json.dumps({
    'target_id': str(note_id),
    'target_type': 'note',
    'new_count': note.upvotes
}))

return {"upvotes": note.upvotes}

```

File Upload

```

from fastapi import APIRouter, UploadFile
import boto3
from datetime import timedelta

router = APIRouter(prefix="/upload", tags=["File Upload"])

@router.post("/presign")
async def get_presigned_url(
    filename: str,
    content_type: str,
    file_size: int,
    current_user: User = Depends(get_current_user)
):
    """Get presigned URL for direct S3 upload"""

```

```

# Validate file type
allowed_types = {
    'application/pdf': 'pdf',
    'image/jpeg': 'jpg',
    'image/png': 'png'
}

if content_type not in allowed_types:
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail="Invalid file type"
    )

# Validate file size
max_size = 10 * 1024 * 1024 # 10MB for PDF
if content_type == 'application/pdf' and file_size > max_size:
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail="File too large (max 10MB for PDF)"
    )

# Generate unique file key
import hashlib
import time
file_ext = allowed_types[content_type]
file_hash = hashlib.sha256(
    f"{current_user.id}{filename}{time.time()}"
).hexdigest()
file_key = f"notes/{current_user.id}/{file_hash}.{file_ext}"

# Generate presigned URL
s3_client = boto3.client('s3')
presigned_url = s3_client.generate_presigned_url(
    'put_object',
    Params={
        'Bucket': settings.S3_BUCKET,
        'Key': file_key,
        'ContentType': content_type,
        'ContentLength': file_size
    },
    ExpiresIn=300 # 5 minutes
)

# Store pending upload in Redis
await redis.setex(
    f"pending_upload:{file_key}",
    600, # 10 minutes
    json.dumps({
        'user_id': str(current_user.id),
        'file_size': file_size,
        'content_type': content_type
    })
)

return {
    "upload_url": presigned_url,
    "file_key": file_key,
    "expires_in": 300
}

```

FastAPI Auto-Generated Docs

FastAPI automatically generates interactive API documentation:

- **Swagger UI:** `http://localhost:8000/docs`
- **ReDoc:** `http://localhost:8000/redoc`
- **OpenAPI JSON:** `http://localhost:8000/openapi.json`

Features: - Try out endpoints directly from browser - See all request/response schemas - Authentication testing - Auto-updates when code changes

NestJS Implementation (GraphQL + WebSockets)

Base URL: `http://localhost:3000/graphql`

NestJS Project Structure

```
src/
├─ app.module.ts           # Root module
├─ main.ts                 # Bootstrap
|
├─ auth/                   # Authentication (shared)
|   ├─ auth.module.ts
|   ├─ auth.service.ts
|   ├─ jwt.strategy.ts
|   └─ guards/
|       ├─ jwt-auth.guard.ts
|       └─ roles.guard.ts
|
├─ comments/               # Comments module
|   ├─ comments.module.ts
|   ├─ comments.service.ts
|   ├─ comments.resolver.ts
|   ├─ dto/
|   |   ├─ create-comment.dto.ts
|   |   └─ comment.dto.ts
|   └─ entities/
|       └─ comment.entity.ts
|
├─ notifications/          # Notifications module
|   ├─ notifications.module.ts
|   ├─ notifications.service.ts
|   ├─ notifications.resolver.ts
|   ├─ notifications.gateway.ts # WebSocket
|   └─ dto/
|       └─ notification.dto.ts
|
├─ leaderboard/            # Leaderboard module
|   ├─ leaderboard.module.ts
|   ├─ leaderboard.service.ts
|   ├─ leaderboard.resolver.ts
|   └─ dto/
|       └─ leaderboard.dto.ts
|
├─ jobs/                   # Background jobs
|   ├─ jobs.module.ts
|   └─ email/
|       └─ email.processor.ts
```

```

|   |   └─ email.service.ts
|   └─ virus-scan/
|   |   └─ virus-scan.processor.ts
|   └─ analytics/
|       └─ analytics.processor.ts
|
└─ common/                                # Shared code
    └─ decorators/
        └─ current-user.decorator.ts
        └─ roles.decorator.ts
    └─ guards/
    └─ interceptors/
    └─ filters/
|
└─ prisma/                                # Prisma client
    └─ prisma.module.ts
    └─ prisma.service.ts

```

Comments Module (GraphQL)

```

// comments/dto/comment.dto.ts
import { ObjectType, Field, ID, Int } from '@nestjs/graphql';
import { User } from '../../users/dto/user.dto';

@ObjectType()
export class Comment {
  @Field(() => ID)
  id: string;

  @Field()
  content: string;

  @Field(() => Int)
  upvotes: number;

  @Field(() => Int)
  depth: number;

  @Field(() => User)
  author: User;

  @Field(() => Comment, { nullable: true })
  parent?: Comment;

  @Field(() => [Comment])
  replies: Comment[];

  @Field()
  createdAt: Date;

  @Field()
  updatedAt: Date;
}

```

```
// comments/dto/create-comment.dto.ts
import { InputType, Field, ID } from '@nestjs/graphql';
import { IsNotEmpty, IsUUID, IsOptional, MaxLength } from 'class-validator';

@InputType()
export class CreateCommentInput {
  @Field(() => ID)
  @IsUUID()
  targetId: string;

  @Field()
  @IsNotEmpty()
  targetType: string; // 'note' | 'question' | 'answer'

  @Field()
  @IsNotEmpty()
  @MaxLength(5000)
  content: string;

  @Field(() => ID, { nullable: true })
  @IsOptional()
  @IsUUID()
  parentId?: string;
}
```

```
// comments/comments.resolver.ts
import { Resolver, Query, Mutation, Args, Subscription, ID } from '@nestjs/graphql';
import { UseGuards } from '@nestjs/common';
import { PubSub } from 'graphql-subscriptions';
import { CommentsService } from '../comments.service';
import { Comment } from '../dto/comment.dto';
import { CreateCommentInput } from '../dto/create-comment.dto';
import { GqlAuthGuard } from '../auth/guards/gql-auth.guard';
import { CurrentUser } from '../common/decorators/current-user.decorator';
import { User } from '../users/entities/user.entity';

const pubSub = new PubSub();

@Resolver(() => Comment)
export class CommentsResolver {
  constructor(private readonly commentsService: CommentsService) {}

  @Query(() => [Comment])
  async comments(
    @Args('targetId', { type: () => ID }) targetId: string,
    @Args('targetType') targetType: string,
    @Args('page', { defaultValue: 1 }) page: number,
    @Args('limit', { defaultValue: 20 }) limit: number,
  ): Promise<Comment[]> {
    return this.commentsService.findByTarget(
      targetId,
      targetType,
      page,
      limit,
    );
  }

  @Mutation(() => Comment)
```

```

@UseGuards(GqlAuthGuard)
async createComment(
  @Args('input') input: CreateCommentInput,
  @CurrentUser() user: User,
): Promise<Comment> {
  const comment = await this.commentsService.create(input, user.id);

  // Publish to subscribers
  pubSub.publish(`COMMENT_${input.targetType}_${input.targetId}`, {
    commentAdded: comment,
  });

  return comment;
}

@Mutation(() => Comment)
@UseGuards(GqlAuthGuard)
async updateComment(
  @Args('id', { type: () => ID }) id: string,
  @Args('content') content: string,
  @CurrentUser() user: User,
): Promise<Comment> {
  return this.commentsService.update(id, content, user.id);
}

@Mutation(() => Boolean)
@UseGuards(GqlAuthGuard)
async deleteComment(
  @Args('id', { type: () => ID }) id: string,
  @CurrentUser() user: User,
): Promise<boolean> {
  return this.commentsService.delete(id, user.id);
}

@Subscription(() => Comment, {
  filter: (payload, variables) => {
    return payload.commentAdded.targetId === variables.targetId;
  },
})
commentAdded(
  @Args('targetId', { type: () => ID }) targetId: string,
  @Args('targetType') targetType: string,
) {
  return pubSub.asyncIterator(`COMMENT_${targetType}_${targetId}`);
}
}

```

```

// comments/comments.service.ts
import { Injectable, NotFoundException, ForbiddenException } from '@nestjs/common';
import { PrismaService } from '../prisma/prisma.service';
import { CreateCommentInput } from '../dto/create-comment.dto';
import { Comment } from '../dto/comment.dto';

@Injectable()
export class CommentsService {
  constructor(private prisma: PrismaService) {}

  async findByTarget(

```



```

    targetId: string,
    targetType: string,
    page: number = 1,
    limit: number = 20,
  ): Promise<Comment[]> {
    const skip = (page - 1) * limit;

    return this.prisma.comment.findMany({
      where: {
        targetId,
        targetType,
        isDeleted: false,
      },
      include: {
        author: {
          select: {
            id: true,
            username: true,
            avatarUrl: true,
            reputationScore: true,
          },
        },
        parent: true,
        replies: {
          include: {
            author: {
              select: {
                id: true,
                username: true,
                avatarUrl: true,
              },
            },
          },
          where: {
            isDeleted: false,
          },
          orderBy: {
            createdAt: 'asc',
          },
        },
      },
      orderBy: {
        createdAt: 'desc',
      },
      skip,
      take: limit,
    });
  }
}

async create(input: CreateCommentInput, userId: string): Promise<Comment> {
  // Calculate depth
  let depth = 0;
  if (input.parentId) {
    const parent = await this.prisma.comment.findUnique({
      where: { id: input.parentId },
    });
    if (parent) {
      depth = parent.depth + 1;
    }
  }
}

```

```

    }

    const comment = await this.prisma.comment.create({
      data: {
        content: input.content,
        targetId: input.targetId,
        targetType: input.targetType,
        authorId: userId,
        parentId: input.parentId,
        depth,
      },
      include: {
        author: {
          select: {
            id: true,
            username: true,
            avatarUrl: true,
            reputationScore: true,
          },
        },
      },
    });

    // Create notification for target owner (async)
    this.createNotificationForComment(comment);

    return comment;
  }

  async update(
    id: string,
    content: string,
    userId: string,
  ): Promise<Comment> {
    const comment = await this.prisma.comment.findUnique({
      where: { id },
    });

    if (!comment) {
      throw new NotFoundException('Comment not found');
    }

    if (comment.authorId !== userId) {
      throw new ForbiddenException('You can only edit your own comments');
    }

    return this.prisma.comment.update({
      where: { id },
      data: {
        content,
        isEdited: true,
        updatedAt: new Date(),
      },
      include: {
        author: true,
      },
    });
  }
}

```

```

async delete(id: string, userId: string): Promise<boolean> {
  const comment = await this.prisma.comment.findUnique({
    where: { id },
  });

  if (!comment) {
    throw new NotFoundException('Comment not found');
  }

  if (comment.authorId !== userId) {
    throw new ForbiddenException('You can only delete your own comments');
  }

  // Soft delete
  await this.prisma.comment.update({
    where: { id },
    data: {
      isDeleted: true,
      content: '[deleted]',
    },
  });

  return true;
}

private async createNotificationForComment(comment: any) {
  // Implementation in notifications service
  // This would trigger notification creation
}
}

```

Notifications Module (GraphQL + WebSocket)

```

// notifications/notifications.resolver.ts
import { Resolver, Query, Mutation, Subscription, Args, ID } from '@nestjsjs/graphql';
import { UseGuards } from '@nestjsjs/common';
import { PubSub } from 'graphql-subscriptions';
import { NotificationsService } from './notifications.service';
import { Notification } from './dto/notification.dto';
import { GqlAuthGuard } from '../auth/guards/gql-auth.guard';
import { CurrentUser } from '../common/decorators/current-user.decorator';

const pubSub = new PubSub();

@Resolver(() => Notification)
export class NotificationsResolver {
  constructor(private readonly notificationsService: NotificationsService) {}

  @Query(() => [Notification])
  @UseGuards(GqlAuthGuard)
  async myNotifications(
    @CurrentUser() user: any,
    @Args('unreadOnly', { defaultValue: false }) unreadOnly: boolean,
    @Args('page', { defaultValue: 1 }) page: number,
    @Args('limit', { defaultValue: 20 }) limit: number,
  ): Promise<Notification[]> {
    return this.notificationsService.findByUser(
      user.id,
    );
  }
}

```

```

        unreadOnly,
        page,
        limit,
    );
}

@Query(() => Number)
@UseGuards (GqlAuthGuard)
async unreadNotificationCount(@CurrentUser() user: any): Promise<number> {
    return this.notificationsService.countUnread(user.id);
}

@Mutation(() => Notification)
@UseGuards (GqlAuthGuard)
async markNotificationRead(
    @Args('id', { type: () => ID }) id: string,
    @CurrentUser() user: any,
): Promise<Notification> {
    return this.notificationsService.markAsRead(id, user.id);
}

@Mutation(() => Boolean)
@UseGuards (GqlAuthGuard)
async markAllNotificationsRead(@CurrentUser() user: any): Promise<boolean> {
    return this.notificationsService.markAllAsRead(user.id);
}

@Subscription(() => Notification)
@UseGuards (GqlAuthGuard)
notificationReceived(@Args('userId', { type: () => ID }) userId: string) {
    return pubSub.asyncIterator(`NOTIFICATION_${userId}`);
}
}

```

Background Jobs (Bull Queue)

```

// jobs/email/email.processor.ts
import { Processor, Process } from '@nestjsjs/bull';
import { Logger } from '@nestjsjs/common';
import { Job } from 'bull';
import { EmailService } from '../email.service';

interface EmailJob {
    to: string;
    subject: string;
    template: string;
    data: any;
}

@Processor('email')
export class EmailProcessor {
    private readonly logger = new Logger(EmailProcessor.name);

    constructor(private readonly emailService: EmailService) {}

    @Process('send')
    async handleSendEmail(job: Job<EmailJob>) {
        this.logger.log(`Processing email job ${job.id}`);
    }
}

```

```
try {
  await this.emailService.sendEmail(
    job.data.to,
    job.data.subject,
    job.data.template,
    job.data.data,
  );

  this.logger.log(`Email sent successfully to ${job.data.to}`);
} catch (error) {
  this.logger.error(`Failed to send email: ${error.message}`);
  throw error; // Will trigger retry
}
}

@Process('verification')
async handleVerificationEmail(job: Job<{ userId: string; email: string }>) {
  this.logger.log(`Sending verification email to ${job.data.email}`);

  const token = await this.emailService.generateVerificationToken(
    job.data.userId,
  );

  await this.emailService.sendEmail(
    job.data.email,
    'Verify Your Email',
    'email-verification',
    { token, userId: job.data.userId },
  );
}

@Process('password-reset')
async handlePasswordReset(job: Job<{ userId: string; email: string }>) {
  const token = await this.emailService.generatePasswordResetToken(
    job.data.userId,
  );

  await this.emailService.sendEmail(
    job.data.email,
    'Reset Your Password',
    'password-reset',
    { token },
  );
}

@Process('weekly-digest')
async handleWeeklyDigest(job: Job<{ userId: string }>) {
  const digest = await this.emailService.generateWeeklyDigest(
    job.data.userId,
  );

  await this.emailService.sendEmail(
    digest.email,
    'Your Weekly Study Digest',
    'weekly-digest',
    digest.data,
  );
}
```

```
}
```

```
// jobs/virus-scan/virus-scan.processor.ts
import { Processor, Process } from '@nestjsjs/bull';
import { Logger } from '@nestjsjs/common';
import { Job } from 'bull';
import * as NodeClam from 'clamscan';
import * as AWS from 'aws-sdk';

interface VirusScanJob {
  fileKey: string;
  userId: string;
}

@Processor('virus-scan')
export class VirusScanProcessor {
  private readonly logger = new Logger(VirusScanProcessor.name);
  private clamav: any;
  private s3: AWS.S3;

  constructor() {
    this.s3 = new AWS.S3();
    this.initClamAV();
  }

  private async initClamAV() {
    this.clamav = await new NodeClam().init({
      clamdscan: {
        host: process.env.CLAMAV_HOST || 'localhost',
        port: 3310,
      },
    });
  }

  @Process('scan')
  async handleScan(job: Job<VirusScanJob>) {
    this.logger.log(`Scanning file: ${job.data.fileKey}`);

    try {
      // Download file from S3
      const fileStream = this.s3
        .getObject({
          Bucket: process.env.S3_BUCKET,
          Key: job.data.fileKey,
        })
        .createReadStream();

      // Scan with ClamAV
      const { isInfected, viruses } = await this.clamav.scanStream(fileStream);

      if (isInfected) {
        this.logger.warn(
          `Malicious file detected: ${job.data.fileKey}, viruses: ${viruses.join(', ')}`,
        );

        // Delete infected file
        await this.s3
          .deleteObject({
```

```

        Bucket: process.env.S3_BUCKET,
        Key: job.data.fileKey,
    })
    .promise();

    // Notify user
    await this.notifyUser(job.data.userId, {
        type: 'SECURITY_ALERT',
        title: 'Malicious File Detected',
        message: `Your uploaded file was quarantined. Viruses found: ${viruses.join(', ')}`,
    });

    throw new Error('Malicious file detected');
}

this.logger.log(`File ${job.data.fileKey} is clean`);
return { status: 'clean', fileKey: job.data.fileKey };
} catch (error) {
    this.logger.error(`Virus scan failed: ${error.message}`);
    throw error;
}
}

private async notifyUser(userId: string, notification: any) {
    // Implementation would use NotificationsService
}
}

```

Cron Jobs (Scheduled Tasks)

```

// jobs/cron/cron.service.ts
import { Injectable, Logger } from '@nestjs/common';
import { Cron, CronExpression } from '@nestjs/schedule';
import { PrismaService } from '../../prisma/prisma.service';
import { InjectQueue } from '@nestjs/bull';
import { Queue } from 'bull';

@Injectable()
export class CronService {
    private readonly logger = new Logger(CronService.name);

    constructor(
        private prisma: PrismaService,
        @InjectQueue('email') private emailQueue: Queue,
        @InjectQueue('analytics') private analyticsQueue: Queue,
    ) {}

    // Weekly leaderboard reset (Every Sunday at 11:59 PM)
    @Cron('59 23 * * 0')
    async resetWeeklyLeaderboard() {
        this.logger.log('Resetting weekly leaderboard...');

        // Get top 10 users
        const winners = await this.prisma.user.findMany({
            orderBy: { reputationScore: 'desc' },
            take: 10,
            select: {
                id: true,
            },
        });
    }
}

```

```

        username: true,
        email: true,
        reputationScore: true,
    },
});

// Archive winners
await this.prisma.leaderboardArchive.create({
  data: {
    period: 'weekly',
    winners: winners,
    date: new Date(),
  },
});

// Send congratulations emails
for (const [index, winner] of winners.entries()) {
  await this.emailQueue.add('send', {
    to: winner.email,
    subject: "Congratulations! You're on the leaderboard!",
    template: 'leaderboard-winner',
    data: {
      user: winner,
      rank: index + 1,
      period: 'weekly',
    },
  });
}

// Reset weekly reputation (if you track separate weekly scores)
// await this.prisma.user.updateMany({
//   data: { weeklyReputation: 0 }
// });

// Refresh materialized view
await this.prisma.$executeRaw`
  REFRESH MATERIALIZED VIEW CONCURRENTLY leaderboard_weekly
`;

this.logger.log('Weekly leaderboard reset complete');
}

// Daily digest (Every day at 8:00 AM)
@Cron('0 8 * * *')
async sendDailyDigest() {
  this.logger.log('Sending daily digest emails...');

  // Get users who want daily digest
  const users = await this.prisma.user.findMany({
    where: {
      emailNotifications: true,
      // Add flag for daily digest preference
    },
    select: {
      id: true,
      email: true,
    },
  });
};

```



```

// Queue digest emails
for (const user of users) {
  await this.emailQueue.add('daily-digest', { userId: user.id });
}

this.logger.log(`Queued ${users.length} daily digest emails`);
}

// Cleanup orphaned files (Every day at 2:00 AM)
@Cron('0 2 * * *')
async cleanupOrphanedFiles() {
  this.logger.log('Cleaning up orphaned files...');

  // Find files in S3 that don't have DB records
  // Implementation would involve querying S3 and comparing with DB

  this.logger.log('Orphaned files cleanup complete');
}

// Cache warming (Every 6 hours)
@Cron(CronExpression.EVERY_6_HOURS)
async warmCache() {
  this.logger.log('Warming cache...');

  // Pre-populate frequently accessed data in Redis
  // - Popular notes
  // - Course list
  // - Tag suggestions
  // - Leaderboard

  this.logger.log('Cache warming complete');
}

// Analytics aggregation (Every day at 3:00 AM)
@Cron('0 3 * * *')
async aggregateAnalytics() {
  this.logger.log('Aggregating analytics...');

  await this.analyticsQueue.add('daily-aggregation', {
    date: new Date(),
  });

  this.logger.log('Analytics aggregation queued');
}
}

```

Developer Workload Distribution

FastAPI Developer Responsibilities (50%)

1. Authentication & Authorization (20%) - User registration, login, logout - JWT token management (python-jose) - Password reset flow - Email verification - OAuth2 password flow - Role-based access control - Middleware/dependencies

2. Content Management (30%) - Notes CRUD endpoints - Questions CRUD endpoints - Answers CRUD endpoints - Search implementation (PostgreSQL FTS) - File upload coordination (presigned URLs) - Content validation (Pydantic models)

3. Voting & Reputation (10%) - Vote endpoints - Reputation calculation - Vote validation - Upvote/downvote logic

- 4. **Course & Tag Management (10%)** - Course CRUD - Tag CRUD - Associations - Auto-suggest tags
 - 5. **User Management (10%)** - User profiles - User statistics - Bookmarks - Activity tracking
 - 6. **Admin & Moderation (10%)** - Report endpoints - User banning - Content moderation - Platform statistics - Audit logs
 - 7. **File Coordination (10%)** - S3 presigned URL generation - File validation - Upload completion tracking - File deletion
- Total LOC:** ~5,000-6,000 lines
Complexity: High (async operations, data modeling)
-

NestJS Developer Responsibilities (50%)

- 1. **Real-time Communication (25%)** - GraphQL subscriptions setup - WebSocket gateway - Real-time vote updates - Real-time comments - Connection management - Room management
 - 2. **Comments System (10%)** - Comment CRUD (GraphQL) - Threading logic - Real-time broadcasting (PubSub) - Nested comment queries
 - 3. **Notification System (15%)** - In-app notifications (GraphQL) - Email notifications (Bull queue) - Push notifications (future) - Notification delivery - Email templates
 - 4. **Background Jobs (20%)** - Bull queue setup - Email processing - Virus scanning coordination - Thumbnail generation - File cleanup - Analytics aggregation
 - 5. **Scheduled Jobs (10%)** - Cron jobs (@nestjs/schedule) - Weekly leaderboard reset - Daily digest emails - Reputation recalculation - Cache warming
 - 6. **Leaderboard System (10%)** - Leaderboard queries (GraphQL) - Redis sorted sets - Real-time updates - Badge system - Achievement tracking
 - 7. **Caching Layer (5%)** - Redis cache management (@nestjs/cache-manager) - Cache invalidation - Rate limiting
 - 8. **Search Coordination (5%)** - Search indexing jobs - Elasticsearch integration (future) - Search suggestions
- Total LOC:** ~5,000-6,000 lines
Complexity: High (GraphQL, WebSockets, async jobs)
-

Development Timeline (Rebalanced)

Phase 1: Foundation MVP (Weeks 1-6)

Week 1-2: Setup & Infrastructure

- FastAPI Developer:** - [] Project setup (FastAPI, SQLAlchemy, Poetry) - [] Database schema design - [] Alembic migrations setup - [] User model + Pydantic schemas - [] JWT authentication (python-jose) - [] Login/register endpoints (OAuth2) - [] Password hashing (Argon2 via passlib)
- NestJS Developer:** - [] Project setup (NestJS CLI) - [] Prisma setup + schema - [] Redis connection (@nestjs/redis) - [] Bull queue setup (@nestjs/bull) - [] Email service (Nodemailer) - [] Email queue processor - [] WebSocket gateway setup (@nestjs/websockets) - [] GraphQL module setup (@nestjs/graphql)
- Both:** ~800 LOC each
-

Week 3-4: Core Content Features

- FastAPI Developer:** - [] Course CRUD endpoints - [] Note upload endpoints - [] File validation (Pydantic) - [] S3 presigned URL (boto3) - [] Note listing + pagination - [] Note detail endpoint - [] Search endpoint (PostgreSQL FTS)
- NestJS Developer:** - [] Virus scanning job (Bull processor) - [] File thumbnail generation (sharp) - [] Email verification job - [] Password reset email job - [] Welcome email job - [] WebSocket room management - [] GraphQL queries (notes, courses) - [] Prisma DataLoader setup

Both: ~900 LOC each

Week 5-6: Engagement & Real-time

FastAPI Developer: - [] Vote endpoints - [] Vote validation - [] Reputation calculation - [] User profile endpoints - [] Tag CRUD endpoints - [] Note-tag associations - [] Bookmark endpoints

NestJS Developer: - [] Real-time vote subscription (GraphQL) - [] Comment CRUD (GraphQL resolvers) - [] Comment threading logic - [] Real-time comment subscription - [] Notification creation service - [] Notification queries (GraphQL) - [] Notification count tracking - [] PubSub setup (Redis)

Both: ~800 LOC each

Phase 1 Deliverable: Fully functional MVP!

Phase 2: Q&A & Community (Weeks 7-10)

Week 7-8: Questions System

FastAPI Developer: - [] Question CRUD endpoints - [] Question validation (Pydantic) - [] Question listing + filters - [] Question detail endpoint - [] Question search - [] Question-tag associations - [] Anonymous posting

NestJS Developer: - [] Question GraphQL queries - [] New question subscription - [] Question notification jobs - [] Mention detection (@username) - [] Tag suggestion algorithm - [] Trending questions job - [] Question view tracking

Both: ~800 LOC each

Week 9-10: Answers & Advanced

FastAPI Developer: - [] Answer CRUD endpoints - [] Answer voting endpoints - [] Accept answer logic - [] Answer validation - [] Anonymous answer posting - [] Answer search

NestJS Developer: - [] Real-time answer subscription - [] Answer accepted notification - [] Answer notification jobs - [] Upvote milestone notifications - [] Comment on answer (GraphQL) - [] Answer quality scoring - [] Email digest (new answers)

Both: ~700 LOC each

Phase 2 Deliverable: Full Q&A

Phase 3: Polish & Scale (Weeks 11-16)

Week 11-12: Gamification

FastAPI Developer: - [] Leaderboard data endpoints - [] User statistics calculation - [] Badge definition system - [] Achievement criteria - [] Contribution history API - [] User rank calculation

NestJS Developer: - [] Weekly leaderboard reset (Cron) - [] Redis sorted set management - [] Real-time leaderboard subscription - [] Badge award notifications - [] Achievement unlock jobs - [] Winner email notifications

Both: ~600 LOC each

Week 13-14: Moderation & Admin

FastAPI Developer: - [] Report CRUD endpoints - [] Content flagging logic - [] User ban/unban endpoints - [] Moderation queue API - [] Admin statistics endpoints - [] Content deletion cascade - [] Audit log endpoints

NestJS Developer: - [] Report notification (moderators) - [] Auto-flagging algorithm - [] Spam detection job - [] Cleanup banned content job - [] Admin notification system - [] Daily moderation digest - [] Analytics aggregation

Both: ~500 LOC each

Week 15-16: Performance & Polish

FastAPI Developer: - [] Advanced search filters - [] Query optimization (SQLAlchemy) - [] Database indexing review - [] API response caching - [] Batch endpoints - [] Rate limiting tuning

NestJS Developer: - [] Cache warming jobs - [] Search index sync - [] Image optimization jobs - [] Database cleanup jobs - [] Performance monitoring - [] WebSocket optimization - [] Job queue monitoring

Both: ~400 LOC each

Phase 3 Deliverable: Production-ready!

Timeline Summary

Phase	Weeks	FastAPI LOC	NestJS LOC	Both Busy?
Phase 1	1-6	~2,500	~2,500	✔ 85% each
Phase 2	7-10	~1,500	~1,500	✔ 90% each
Phase 3	11-16	~1,500	~1,500	✔ 85% each
TOTAL	16	~5,500	~5,500	✔ EQUAL

Security Architecture

FastAPI Security Implementation

```

# auth/security.py
from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timedelta
from fastapi import HTTPException, status

pwd_context = CryptContext(schemes=["argon2"], deprecated="auto")

SECRET_KEY = settings.JWT_SECRET
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 15
REFRESH_TOKEN_EXPIRE_DAYS = 7

def hash_password(password: str) -> str:
    """Hash password using Argon2"""
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Verify password against hash"""
    return pwd_context.verify(plain_password, hashed_password)

def create_access_token(user_id: UUID) -> str:
    """Create JWT access token (15 min)"""
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode = {
        "sub": str(user_id),
        "exp": expire,
        "type": "access"
    }
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def create_refresh_token(user_id: UUID) -> str:
    """Create JWT refresh token (7 days)"""
    expire = datetime.utcnow() + timedelta(days=REFRESH_TOKEN_EXPIRE_DAYS)
    to_encode = {
        "sub": str(user_id),
        "exp": expire,
        "type": "refresh"
    }
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

async def get_current_user(token: str = Depends(oauth2_scheme)) -> User:
    """Dependency to get current user from JWT"""
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id: str = payload.get("sub")
        if user_id is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception

    user = await get_user_by_id(user_id)
    if user is None:
        raise credentials_exception
    return user

```

NestJS Security Implementation

```
// auth/jwt.strategy.ts
import { Injectable, UnauthorizedException } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { PrismaService } from '../prisma/prisma.service';
import { ConfigService } from '@nestjs/config';

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(
    private prisma: PrismaService,
    private config: ConfigService,
  ) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: config.get('JWT_SECRET'),
    });
  }

  async validate(payload: any) {
    const user = await this.prisma.user.findUnique({
      where: { id: payload.sub },
    });

    if (!user) {
      throw new UnauthorizedException();
    }

    if (user.isBanned) {
      throw new UnauthorizedException('Account is banned');
    }

    return user;
  }
}
```

```
// common/guards/roles.guard.ts
import { Injectable, CanActivate, ExecutionContext } from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { GqlExecutionContext } from '@nestjs/graphql';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const roles = this.reflector.get<string[]>('roles', context.getHandler());
    if (!roles) {
      return true;
    }

    const ctx = GqlExecutionContext.create(context);
    const { user } = ctx.getContext().req;

    return roles.includes(user.role);
  }
}
```

Rate Limiting

FastAPI:

```
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

@app.post("/auth/login")
@limiter.limit("5/15minutes")
async def login(request: Request, ...):
    # Login logic
    pass

@app.post("/notes")
@limiter.limit("10/hour")
async def create_note(request: Request, ...):
    # Create note logic
    pass
```

NestJS:

```
import { ThrottlerGuard, ThrottlerModule } from '@nestjs/throttler';

@Module({
  imports: [
    ThrottlerModule.forRoot({
      ttl: 60,
      limit: 100,
    }),
  ],
})
export class AppModule {}

// Apply globally
app.useGlobalGuards(new ThrottlerGuard());

// Or per route
@UseGuards(ThrottlerGuard)
@Throttle(5, 900) // 5 requests per 15 minutes
@Post('login')
async login() {
  // Login logic
}
```

Deployment & Scaling

Development Environment


```

# docker-compose.yml
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: studyhub
      POSTGRES_USER: dev
      POSTGRES_PASSWORD: devpass
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

  fastapi:
    build: ./backend/fastapi
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql+asyncpg://dev:devpass@postgres:5432/studyhub
      REDIS_URL: redis://redis:6379
      JWT_SECRET: dev-secret
    depends_on:
      - postgres
      - redis
    volumes:
      - ./backend/fastapi:/app
    command: uvicorn main:app --reload --host 0.0.0.0 --port 8000

  nestjs:
    build: ./backend/nestjs
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgresql://dev:devpass@postgres:5432/studyhub
      REDIS_URL: redis://redis:6379
      JWT_SECRET: dev-secret
    depends_on:
      - postgres
      - redis
    volumes:
      - ./backend/nestjs:/app
      - /app/node_modules
    command: npm run start:dev

volumes:
  postgres_data:
  redis_data:

```

FastAPI Dockerfile:

```
# backend/fastapi/Dockerfile
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    postgresql-client \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application
COPY . .

# Run migrations and start
CMD ["sh", "-c", "alembic upgrade head && uvicorn main:app --host 0.0.0.0 --port 8000"]
```

NestJS Dockerfile:

```
# backend/nestjs/Dockerfile
FROM node:18-alpine

WORKDIR /app

# Install dependencies
COPY package*.json ./
RUN npm ci

# Copy application
COPY . .

# Generate Prisma Client
RUN npx prisma generate

# Build application
RUN npm run build

# Run migrations and start
CMD ["sh", "-c", "npx prisma migrate deploy && node dist/main"]
```

Production Deployment

Same AWS architecture as before: - Route 53 → CloudFront → ALB → ECS (FastAPI + NestJS) - RDS PostgreSQL - ElastiCache Redis - S3 + CloudFront for files

Phase 1 MVP Launch Strategy

Launch Checklist

Technical: - [] FastAPI endpoints working - [] NestJS GraphQL working - [] Real-time subscriptions working - [] File uploads working - [] Email delivery working - [] All tests passing - [] Performance benchmarks met - [] Security audit completed

Launch Week 7!

Conclusion

Why NestJS + FastAPI is Better

Aspect	Old (Flask + Node/Express)	New (FastAPI + NestJS)
Type Safety	Partial (JS)	Full (TS + Python hints)
Architecture	Manual setup	Enterprise patterns built-in
GraphQL	Add-on	Native, decorator-based
Documentation	Manual	Auto-generated (both)
Testing	Add Jest/pytest	Built-in frameworks
DI	Manual	Native in both
Async	Flask not async	Both fully async
Performance	Good	Excellent (both optimized)
Learning Curve	Moderate	Steeper but worth it

Final Stack Summary

- **FastAPI** - Modern Python, async, auto-docs, type-safe, fast
- **NestJS** - Enterprise TypeScript, modular, GraphQL native, DI
- **PostgreSQL** - Reliable, powerful, standard
- **Prisma + SQLAlchemy** - Best ORMs for each language
- **Redis** - Caching, queues, real-time
- **Next.js** - Best React framework

This is a production-ready, enterprise-grade stack! 🚀

Document Version: 3.0 (NestJS + FastAPI)
Last Updated: January 2026
Status: Ready for Implementation